

VULNERABILITY DISCLOSURE REPORT

WAF Path Normalization & Filter Bypass

Classification: CONFIDENTIAL

Report Date: May 05, 2026

Researcher: Amin Hisham Shembesh

1. Researcher Information

Field	Details
Full Name	Amin Hisham Shembesh
Role / Title	Junior Penetration Tester & SOC Analyst
Education	B.S. Candidate, Computer Science & Engineering (Cybersecurity)
LinkedIn	linkedin.com/in/amin-shembesh-4aab5a250
Disclosure Type	Responsible / Coordinated Disclosure

2. Executive Summary

During an independent security assessment, a logic-level vulnerability was identified in a commercial Web Application Firewall (WAF) product. The flaw relates to insufficient URL path normalization prior to signature matching and rule enforcement.

The WAF fails to canonicalize certain URL path sequences before evaluating incoming requests against its security rule set. As a result, an attacker can craft specially formed URLs that bypass WAF protections entirely, while the underlying backend server still processes the request normally. This represents a fundamental failure in the WAF's primary function as a perimeter defense layer.

Two specific bypass techniques were confirmed:

- Double-slash prefix injection (e.g., `//env`, `//config`) — bypasses path-based WAF rules.
- Semicolon-dot path injection (e.g., `/./admin`, `/endpoint/./status`) — bypasses access control filters.

This report focuses exclusively on the logic flaw within the WAF product itself. All testing was conducted as part of independent security research, and this disclosure is made in good faith to assist in improving product security.

3. Finding Summary

Finding	Severity	Component Affected	Status
Path Normalization Bypass	HIGH	WAF Rule Engine	Unpatched
Double-Slash Filter Evasion	HIGH	WAF URL Parser	Unpatched
./; Sequence Bypass	HIGH	WAF Signature Matching	Unpatched

4. Detailed Technical Findings

4.1 Double-Slash Path Normalization Bypass

Severity: HIGH

CVE Reference: None assigned (awaiting vendor confirmation)

CWE: CWE-20: Improper Input Validation / CWE-706: Use of Incorrectly-Resolved Name or Reference

Description:

The WAF does not collapse redundant forward slashes (//) in URL paths before applying its security signature rules. HTTP clients and backend web servers normalize // to / during request parsing, meaning a request such as //.env is treated by the backend as a request for /.env. However, because the WAF evaluates the raw, un-normalized path, its rules designed to block access to /.env are never triggered.

Proof of Concept:

Standard blocked request (WAF intercepts correctly):

```
curl -I https://[protected-host]/.env
```

Result: 403 Forbidden (WAF blocks the request)

Bypass request (WAF fails to intercept):

```
curl -I https://[protected-host]//.env
```

Result: 301 Moved Permanently OR backend response (WAF bypassed)

Root Cause:

The WAF's URL parser evaluates the raw request path string directly against its rule signatures without first applying path normalization (collapsing //, resolving ../, removing trailing dots, etc.). This disconnect between how the WAF evaluates a path and how downstream servers resolve it creates the bypass window.

4.2 Semicolon-Dot Sequence Bypass (./; Injection)

Severity: HIGH

CVE Reference: None assigned (awaiting vendor confirmation)

CWE: CWE-20: Improper Input Validation / CWE-284: Improper Access Control

Description:

The WAF does not correctly handle the ./; character sequence in URL paths. Certain backend server technologies normalize the sequence ./; by stripping the semicolon and treating the path segment as a standard path separator. For example, /manager./;/status may be interpreted by such a backend as /manager/status. The WAF, however, does not recognize this normalization behavior and fails to match rules protecting those paths against the obfuscated form, allowing the request to pass through uninspected.

Proof of Concept:

Standard blocked request (WAF intercepts correctly):

```
GET /manager/status HTTP/1.1
```

Result: 403 Forbidden (WAF blocks the request)

Bypass request (WAF fails to intercept):

```
GET /manager./;/status HTTP/1.1
```

Result: 200 OK (WAF bypassed, backend resolves path and grants access)

Root Cause:

The WAF's path parser does not strip or normalize semicolon-separated path parameters before rule matching. RFC 3986 defines that path parameters (content following a semicolon within a path segment) are legal but optional. Backend servers may process these in ways that effectively alter the resolved path, and the WAF must account for this behavior during normalization.

5. Impact Assessment

The ability to bypass the WAF's rule engine has the following potential consequences for any deployment of this product:

- Sensitive file exposure: Attackers may confirm the existence of, and in some cases retrieve, sensitive configuration files such as .env, .git/config, wp-config.php, and database credential files that WAF rules are intended to protect.
- Administrative interface access: On certain backend server configurations, the /./ bypass may expose management and deployment interfaces to unauthenticated attackers.
- Security control nullification: Client environments that rely on the WAF as a primary or sole security layer are exposed to a wide range of web-layer attacks that would normally be blocked, including path traversal, LFI, and targeted reconnaissance.
- Bypass of virtual patching: Organizations using the WAF for virtual patching of known vulnerabilities may have those protections rendered ineffective for any attack vector representable using these obfuscation techniques.

6. Recommended Remediation

6.1 Implement Pre-Matching URL Normalization

Before any security rule is evaluated, the WAF should normalize the incoming request URL through the following canonicalization steps:

- Collapse consecutive forward slashes: Reduce all occurrences of // (and ///+) to a single /.
- Resolve dot-segments: Process all ../ (current directory) and ../../ (parent directory traversal) sequences to their canonical resolved paths.
- Strip path parameters: Remove semicolon-separated path parameters (e.g., /path;param=value should normalize to /path) prior to rule matching.
- Decode percent-encoded characters: Fully decode percent-encoded characters (e.g., %2F, %2E) before normalization and rule evaluation to prevent encoding-based bypasses.
- Normalize backslashes: Convert any backslash (\) characters in URL paths to forward slashes, as some backends accept both.

6.2 Align WAF Path Parsing with Backend Server Behavior

The normalization logic within the WAF should be designed to mirror the path resolution behavior of the specific backend server technologies it is protecting. WAF testing should validate that a blocked request remains blocked when common obfuscation variants of that path are submitted.

6.3 Regression Testing Suite

A regression test suite should be developed and maintained that specifically tests path normalization bypass scenarios, including at minimum:

- Double-slash variations: //, ///, and combinations with encoded slashes (%2F).

- Dot-segment variations: `./`, `/foo/./`, and combinations thereof.
- Semicolon path parameters: `/path;x=y`, `/path/./subpath`.
- Mixed encoding and obfuscation: Combinations of the above techniques.

6.4 Defense-in-Depth Guidance

While a product-level fix is being developed, the following interim mitigations are recommended:

- Configure backend web servers to explicitly normalize and reject malformed URL paths at the server level.
- Restrict access to sensitive paths and administrative interfaces by source IP address or require VPN-based authentication, independent of WAF rules.
- Enable detailed access logging on backend servers to detect bypass attempts during the interim period.

7. Disclosure Timeline

Date	Event
May 05, 2026	Vulnerability confirmed through independent security research.
May 05, 2026	Initial disclosure report submitted to the vendor security team.
Pending	Vendor acknowledgment and triage.
Pending	Patch development, testing, and release.

8. Closing Statement

This disclosure was prepared and submitted in accordance with responsible disclosure principles. All findings described in this report were obtained during independent security research. No unauthorized access to systems, data, or infrastructure was performed or intended.

Strict confidentiality has been maintained regarding these findings and no information has been shared with any third party. The goal of this report is solely to assist in improving the security of the WAF product and to protect the end users and organizations that depend on it.

The researcher is available to collaborate with the vendor security team during the remediation and validation process and can answer any technical questions or provide additional evidence as required.

Amin Hisham Shembesh

Junior Penetration Tester & SOC Analyst

B.S. Candidate, Computer Science & Engineering — Cybersecurity

LinkedIn: [linkedin.com/in/amin-shembesh-4aab5a250](https://www.linkedin.com/in/amin-shembesh-4aab5a250)

Date: May 05, 2026